

Universidad Gerardo Barrios

Facultad de Ciencia Y Tecnología.

Ingeniería en Sistemas y Redes de Informática.



Universidad
Gerardo Barrios

Desarrollo de aplicaciones con Inteligencia Artificial – Implementación y Escalabilidad

Nombre del proyecto

**Herramienta Autónoma para la Gestión de Ahorros y Presupuestos
Personales**

Nombre de la Aplicación

Ex-Codox

Presentado por:

SMIS013221 - Guevara Jiménez, Emely Alexandra

SMIS083121 - Luna Acosta, Kevin Alexander.

SMIS048021 - Trigueros Blanco, Natalia Alexandra

San Miguel, julio de 2026

Descripción breve del servicio preparado

Finance AI API es un backend en FastAPI para análisis financiero personal. Integra autenticación con Firebase, persistencia en Firestore y capacidades de IA con OpenAI (resumen, análisis, recomendaciones, predicción, clasificación, chat y OCR).

Estado actual:

- Estructura del servicio analizada.
- Entrypoint confirmado en [app/main.py](#).
- Endpoint de salud confirmado en [app/routers/health.py](#).
- Endpoints principales de IA confirmados en [app/routers/ai.py](#).

Ruta elegida

Ruta elegida: Docker local (contenedor reproducible en entorno aislado).

Justificación:

- Permite ejecución consistente fuera del entorno local.
- Es la ruta más simple y apropiada para Semana 4.
- Facilita migración posterior a PaaS o cloud.

Dockerfile o configuración de despliegue equivalente

Archivo preparado: [Dockerfile](#)

Resumen de la configuración:

- Base: python:3.11-slim-bookworm (Debian).
- Directorio de trabajo: /app.
- Instalación de dependencias desde requirements.txt.
- Copia de código runtime (carpeta app).
- Exposición del puerto 8000.
- Inicio con Uvicorn en 0.0.0.0:8000.

.dockerignore o explicación de exclusiones

Archivo preparado: [.dockerignore](#)

Exclusiones relevantes:

- Entornos virtuales y cachés: `.venv/`, `venv/`, **`pycache/`**, `*.pyc`.
- Herramientas de test/lint: `.pytest_cache/`, `.ruff_cache/`.
- Control de versiones y editor: `.git/`, `.github/`, `.vscode/`.
- Secretos: `.env`, `.env.*`, `credentials/`.
- Archivos no runtime: `tests/`.

Objetivo:

- Reducir tamaño de build context.
- Evitar inclusión accidental de secretos en la imagen.

Variables de entorno documentadas mediante `.env.example` o tabla equivalente

Archivo de ejemplo: [.env.example](#) Archivo local real (no versionar): [.env](#)

Tabla de variables:

- `APP_NAME`: Nombre de la app.
- `APP_VERSION`: Versión de la app.
- `ENVIRONMENT`: Entorno (development, staging, production).
- `HOST`: Host lógico de configuración.
- `PORT`: Puerto lógico de configuración.
- `CORS_ORIGINS`: Orígenes permitidos por CORS.
- `OPENAI_API_KEY`: Clave de OpenAI (secreto).
- `OPENAI_MODEL`: Modelo principal de IA.
- `OPENAI_MAX_OUTPUT_TOKENS`: Límite de salida del modelo.
- `OPENAI_OCR_MODEL`: Modelo de OCR.
- `OCR_MAX_FILE_SIZE_MB`: Límite de tamaño de imagen para OCR.

Nota de seguridad:

- No publicar valores reales de secretos.
- Mantener .env fuera de Git.

Evidencia de construcción, ejecución o despliegue

Estado actual:

- Validación de compose realizada con éxito: docker compose config.
- Archivo preparado: [docker-compose.yml](#).

Construcción:

- Ejecutar: docker compose build

```
> docker compose build
[+] Building 1.2s (12/12) FINISHED
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 515B 0.0s
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 316B 0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim-bookworm 1.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 170B 0.0s
=> [1/5] FROM docker.io/library/python:3.11-slim-bookworm@sha256:d29f48a31a 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 3.44kB 0.0s
=> CACHED [2/5] WORKDIR /app 0.0s
=> CACHED [3/5] COPY requirements.txt . 0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app ./app 0.0s
=> exporting to image 0.0s
=> => exporting layers 0.0s
=> => writing image sha256:6d24eee0040dbd01786178740c574be1962c222b840b5825 0.0s
=> => naming to docker.io/library/finance-ai-api:week4 0.0s
=> resolving provenance for metadata file 0.0s
[+] Building 1/1
✓ finance-ai-api:week4 Built 0.0s
```

2. Ejecución:

- Ejecutar: docker compose up -d

```
> docker compose up -d
[+] Running 2/2
✓ Network finance_ai_api_default Create... 0.1s
✓ Container finance-ai-api Started 0.3s
> docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
TUS	PORTS	NAMES		
0785a98735be	finance-ai-api:week4	"uvicorn app.main:ap..."	24 seconds ago	Up
23 seconds	0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp	finance-ai-api		

3. Logs:

- Ejecutar: `docker logs finance-ai-api`

```
> docker logs finance-ai-api
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: 192.168.110.121:51084 - "GET /docs HTTP/1.1" 200 OK
INFO: 192.168.110.121:51084 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 192.168.110.232:60806 - "GET /data/financial/2s5ajLyIt4NlieTDYOSU3a0jWPG3 HTTP/1.1" 200 OK
2026-08-09 00:06:07,615 | INFO | HTTP Request: POST https://api.openai.com/v1/responses "HTTP/1.1 200 OK"
INFO: 192.168.110.232:41128 - "POST /ai/recommend/2s5ajLyIt4NlieTDYOSU3a0jWPG3 HTTP/1.1" 200 OK
INFO: 192.168.110.232:41138 - "GET /health/ HTTP/1.1" 200 OK
2026-08-09 00:06:14,956 | INFO | HTTP Request: POST https://api.openai.com/v1/responses "HTTP/1.1 200 OK"
INFO: 192.168.110.232:41128 - "POST /ai/summary/2s5ajLyIt4NlieTDYOSU3a0jWPG3 HTTP/1.1" 200 OK
2026-08-09 00:06:24,234 | INFO | HTTP Request: POST https://api.openai.com/v1/responses "HTTP/1.1 200 OK"
INFO: 192.168.110.232:41128 - "POST /ai/analyze/2s5ajLyIt4NlieTDYOSU3a0jWPG3 HTTP/1.1" 200 OK
INFO: 192.168.110.232:56464 - "POST /ai/recommend/2s5ajLyIt4NlieTDYOSU3a0jWPG3 HTTP/1.1" 200 OK
```

Prueba del endpoint /health

Endpoint esperado: GET /health/

Comando sugerido:

- `curl -L http://localhost:8000/health/`

Evidencia a adjuntar:

```
> curl -L http://192.168.110.160:8000/health
{"success":true,"message":"API funcionando correctamente.,"data":{"status":"running","firebase":"Connected","openai":"Configured","version":"1.0.0","timestamp":"2026-08-09T00:09:31.861132Z"}}%
~/Finance_AI_API
```

Prueba del endpoint principal del proyecto

Endpoint elegido: POST /ai/chat Referencia: [app/routers/ai.py](#)

Requisitos:

- Header Authorization con Firebase ID token válido.
- uid del body debe coincidir con el uid autenticado del token.

Evidencia a adjuntar:

- Método HTTP: POST.
- Body enviado.

[Edit Value](#) | [Schema](#)

```
{
  "uid": "2s5ajLy1t4N1ieTDYOSU3a0jWPG3",
  "question": "¿Cómo puedo ahorrar más dinero según mi situación financiera?"
}
```

- Respuesta recibida.

200

Response body

```
{
  "success": true,
  "message": "Respuesta generada correctamente.",
  "data": {
    "uid": "2s5ajLy1t4N1ieTDYOSU3a0jWPG3",
    "question": "¿Cómo puedo ahorrar más dinero según mi situación financiera?",
    "answer": "Tu ahorro actual es bueno: $219.84, equivalente al 52.4% de tus ingresos. Para ahorrar más, separa ese monto apenas recibas ingresos y evita usarlo. Controla especialmente 'services', tu categoría de mayor gasto, con $142.75 en la cuota de la U. Reserva ese valor con anticipación y revisa si puedes reducir otros gastos menor es antes de comprometer tu balance disponible."
  }
}
```

 [Download](#)

Response headers

```
access-control-allow-credentials: true
content-length: 579
content-type: application/json
date: Sun, 09 Aug 2026 00:13:38 GMT
server: uvicorn
```

Responses

Code	Description	Links
------	-------------	-------

Errores, intentos, bloqueos y correcciones realizadas

Registro inicial:

- Intento: Validación de docker-compose.
- Resultado: Exitoso.
- Comando: docker compose config.

Posibles bloqueos identificados y corrección prevista:

1. Error de credenciales Firebase no encontradas.
 - Causa: El runtime busca /app/credentials/firebase-admin.json.
 - Corrección: Montaje seguro en volumen de solo lectura desde host.
2. Error 401 en endpoints protegidos.
 - Causa: Token inválido/expirado o ausente.
 - Corrección: Regenerar Firebase ID token e incluir Bearer válido.
3. Error 403 por uid distinto.
 - Causa: uid del body no coincide con el uid del token.
 - Corrección: Usar uid del usuario autenticado.

Plan de infraestructura mínima

Objetivo Semana 4 (mínimo viable):

- 1 contenedor API (FastAPI + Uvicorn).
- Variables por archivo .env local (no versionado).
- Credencial Firebase montada como volumen read-only.
- Red por defecto de Docker Compose.
- Puerto publicado 8000:8000.

Evolución sugerida (siguiente fase):

- Gestor de secretos (cloud secret manager).
- Reverse proxy con TLS.
- Observabilidad básica (logs centralizados, health checks).

Estimación de costos iniciales con supuestos

Escenario local (actual):

- Docker en equipo local: costo directo 0 USD.
- OpenAI: costo variable por consumo de tokens.
- Firebase: depende del plan y del volumen (lecturas/escrituras).

Escenario nube básico (referencial):

- 1 instancia pequeña para API: 5 a 15 USD/mes.
- Tráfico bajo y uso moderado de Firestore/Auth: 0 a 20 USD/mes.
- Consumo OpenAI inicial (uso académico/pruebas): 5 a 30 USD/mes.

Supuestos:

- Tráfico bajo (pruebas y demo).
- Sin alta disponibilidad.
- Sin CDN ni balanceador dedicado.

Riesgos técnicos pendientes

- Dependencia de archivo local de Firebase Admin en runtime.
- Dependencia de token Firebase válido para pruebas funcionales protegidas.
- Variabilidad de costo en OpenAI por prompts y frecuencia de uso.
- Falta de pipeline CI/CD en etapa actual.
- Sin estrategia de rotación automática de secretos.